

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	Total Marks:	100
CO2 Statement:	<i>Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)</i>		
Student Name:	C. Shahanaj	Reg. No.:	192372242
Section / Batch:	2023 / CSE-A1	Date:	25/06/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems. • Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required. • Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis. • Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Focus Area	Questions
---------------	------------	-----------

Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis	Q1
------------------------------------	--	----

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I Shaharaj - 198372292 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Shaharaj

RUBRICS FOR CALCULATION

Criteria	Wt .	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20 %	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30 %	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts

Problem-Solving Approach	20 %	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20 %	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10 %	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%				
Application of Concepts	30%				
Problem-Solving Approach	20%				
Accuracy of Solution	20%				
Clarity	10%				

Total Marks (100):	
---------------------------	--

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Selection Sort Duplicate Large Values Analysis:

Problem Interpeltation:- We need to sort the dataset $[31, 48, 12, 48, 7, 25]$ (size $N=6$) in non-decreasing order using selection sort, taking special note of how repeated large values (48) behaves.

Suitability :- Selection sort is a straightforward brute-force comparison based algorithm. It scans the unsorted portion of the array in every pass to find the absolute minimum and swaps it into its correct position.

ii) Step-by-step Sorting Execution:

initial Array :- $[31, 48, 12, 48, 7, 25]$

Pass 1 :- $[31, 48, 12, 48, 7, 25]$
 $\downarrow$ $\downarrow$
 cp min

Action :- $31 > 7$ at index of 0 & 4
swap with index 0 (7) &
increment by 1

Pass 2:- [7, 48, 12, 48, 31, 25]

48 > 12 at index of 1 and 2
swap with index 1 and
increment by 1

Pass 3:- [7, 12, 48, 48, 31, 25]

48 > 25 at index of 2 & 5
swap with index 2 with 25
increment by 1

Pass 4:- [7, 12, 25, 48, 31, 48]

48 > 31 at index of 2 & 3
swap with index 3 with 31 &
increment by 1

pass 5: [7, 12, 25, 31, 48, 48]

swap element at index 4 with itself (or no-op depending on implementation)

iii) Comparison and swap counts:

Total comparisons:

$$\sum_{i=1}^{N-1} i = 5 + 4 + 3 + 2 + 1 = 15 \text{ comparisons}$$

Total swaps

$$N-1 = 5$$

iv) Time complexity and Behavior Analysis:

Time complexity:-

- Best case: $O(N^2)$ - comparisons are made regardless of array order
- Average case: $O(N^2)$
- Worst case: $O(N^2)$

Behavior on Duplicates:-

- Selection sort is Unstable.
- The two 48 values: the first 48 originally at index 1 was swapped to the back (index 5) in pass 3. The second 48 ended up at index 4.

2 Binary Search - First Element Detection:

Problem Interpretation:- Search for the target value 3 in the pre-sorted dataset $[3, 8, 14, 19, 25, 31, 40]$ (Size $N=7$)

Suitability:- Binary search requires a sorted dataset. Because the array is already sorted in ascending order, Binary search allows to eliminate half of the remaining elements in each comparison step, achieving $O(\log N)$ performance instead of scanning sequentially ($O(N)$).

ii) Step-by-step search Execution:

initial Array: [3, 8, 14, 19, 25, 31, 40]

Target: 3

indices: Low = 0, high = 6

iteration 1:-

calculate Mid

$$\text{mid} = \left\lfloor \frac{\text{low} + \text{high}}{2} \right\rfloor = \left\lfloor \frac{0 + 6}{2} \right\rfloor$$

$$= 3$$

Element at mid: array[3] = 19

Comparison 1: target (3) with array[3] (19)

- $3 < 19$, so target must be in the left

- Update interval:- high = mid - 1 = 2.

New search interval is indices [0..2] ([3, 8, 14]).

iteration 2:-

calculate Mid:-

$$\text{Mid} = \left\lfloor \frac{0 + 2}{2} \right\rfloor = 1$$

Element at Mid:- array[1] = 8

Comparison 2:- Compare target (3) with array[1] (8).

iteration 3:-

calculate Mid:

$$\text{Mid} = \left\lfloor \frac{0 + 0}{2} \right\rfloor = 0$$

Element at Mid:- array[0] = 3

Comparison 3:- Compare target (3) with array[0] (3)

iii) Comparison Count & Complexity Analysis:

Total Comparison: 3 comparisons.

Time complexity:-

- Best case:- $O(1)$
- Average case:- $O(\log N)$
- Worst case:- $O(\log N)$

For $N=7$, $\lceil \log_2(7) \rceil + 1 = 3$ maximum comparisons.

Verification & First-element performance Interpretation:-

Verification:- The target 3 was successfully located at index 0.

Interpretation:-

- Even though the element 3 is at the very beginning of the array (index 0), standard Binary search does not find it in 1 step because it always begins checking at the middle of the current sub-array.

$mid=3 \rightarrow mid=1 \rightarrow mid=0$

- Finding an element at the boundary requires repeatedly halving the search space all the way down to the edge, taking $\approx \log_2 N$ steps.
- Linear search would have found 3 in 1 comparison, showing that while Binary search guarantees $O(\log N)$ overall, linear search can outperform it for targets located at index 0.